MyCodeSchool

React — fișă rapidă

User Management fără server · cum gândești fiecare exercițiu · Rareș

1. Cum lucrezi

```
npm install      o dată, după clonare
npm run dev      ──► http://localhost:5173 · Ctrl+C
F12 → Console    erorile apar aici

mockup/index.html ──copiezi HTML──► UserList.tsx
                    class    ──►  className
                    <br>     ──►  <br />
```

```
scrii DOAR în src/users/UserList.tsx

un exercițiu ──► salvezi ──► te uiți în browser
                          ──► „Pe ecran” / „Gata când” din fișă?
                                da ──► commit ──► următorul
                                nu ──► F12, citești eroarea
```

2. Harta componentei (unde pui fiecare lucru)

```
import { useState } from "react";                                          ex. 2
type User = { id, firstName, lastName, email, age };                       ex. 1   deasupra funcției

export default function UserList() {
  ┌ STARE ──────────── users · firstName · lastName · email · age · nextId · editedId   ex. 2, 7, 8
  ├ CALCULATE ──────── canSave (· filtered · minors · averageAge · emailError la bonus) ex. 9
  ├ FUNCȚII ────────── remove(id) · save() · edit(user)                    ex. 6, 7, 8
  └ return ( JSX ) ── titlu (users.length) · formular · tabel (map)        ex. 3, 4, 7
}
```

3. O listă nu se modifică, se înlocuiește

```
users = [ Ana, Radu, Ioana ]

șterg Radu    filter( u.id !== id )
              ──► [ Ana, Ioana ]

adaug Mihai   [ ...users, mihai ]
              ──► [ Ana, Radu, Ioana, Mihai ]

editez Radu   map( u.id === id ? nou : u )
              ──► [ Ana, Radu*, Ioana ]

apoi mereu:   setUsers( lista nouă ) ──► redesenare
```

```
click „Sterge” pe Radu
   │  onClick={() => remove(user.id)}
   ▼
remove(2) ──► filter ──► setUsers([...])
   │
   ▼
React redesenează ──► titlul (4), tabelul fără Radu

users.push(x)   ✗ aceeași listă, React nu vede
users = [...]   ✗ nu e setUsers, nu redesenează
```

4. Formularul: adaug sau editez?

```
fiecare câmp = o stare (input controlat)

 tastezi ──► onChange ──► setFirstName(...)
    ↑                          │
    └──────── value={firstName} ←┘

vârsta: din input vine text ──► Number(...)
nextId: stare, nu let (let se pierde la redesenare)
```

```
„Editeaza” ──► edit(user) ──► editedId = user.id
                          └──► câmpurile = datele lui

„Salveaza” ──► save()
                 editedId === 0 ?
                   da ──► adaug (ex. 7)
                   nu ──► înlocuiesc (ex. 8)
               ──► golesc câmpurile, editedId = 0
```

5. Pas cu pas

1	tipul `User`	5 câmpuri; text = `string`, numere = `number`
2	lista în stare	importul sus; `useState<User[]>([ {...}, {...} ])`; 5 obiecte cu toate cele 5 câmpuri
3	câți sunt	o listă are `.length`; o valoare în JSX stă în `{ }`
4	tabelul	copiezi UN `<tr>` din mockup, îl pui în `users.map(...)`, cu `key={user.id}`; inițialele: `user.firstName[0]`
5	minor / adult	`? :` direct în `className`; granița: 17 e minor, 18 e adult
6	șterge	schema din 3; pe buton funcție săgeată, ca să trimiți `user.id`
7	adaugă	o stare per câmp + `nextId`; în `save()`: obiect nou → `[...users, user]` → `nextId + 1` → golești
8	editează	`editedId` (0 = adaug); `edit()` umple câmpurile; în `save()` alegi între 7 și `map`
9	nu salva goluri	`const canSave: boolean = ... && ...`; pe buton `disabled={!canSave}`
B	bonus	căutare: `filter` + `toLowerCase().includes()`, tabelul face `map` pe rezultat · medie: `reduce` / `length` · confirmare: `useState<User | null>(null)` · erori: mesaj calculat cu `? :`, `is-invalid` cu `? :`, mesajul cu `&&`

6. Capcane

`onClick={remove(user.id)}`	șterge tot la desenare → `onClick={() => remove(user.id)}`
`users.push(user)`	nu redesenează → `setUsers([...users, user])`
`let nextId = 6`	revine la 6 la fiecare redesenare → id-uri dublate → `useState`
`setAge(e.target.value)`	e text, nu număr → `Number(e.target.value)` + `type="number"`
fără `key`	avertisment în consolă; rândurile se pot încurca la ștergere
după salvare	câmpurile rămân pline → le golești în `save()`